A Quorum-Based Total Order Broadcast Protocol for Distributed Intelligence Databases

Bianchi Francesco, Marostica Alessandro

Distributed Systems 1 Project – August 25, 2026

1 Project Structure

The system is a replicated array of integers kept consistent by a two-phase total order broadcast, coordinated by one distinguished replica. When the coordinator crashes, the surviving replicas elect a new one with a ring-based algorithm and resume service in a new epoch.

Actors

There are three kinds of actor. `Replica` holds a local copy of the array and implements the whole protocol: it serves reads, forwards writes to the coordinator, participates in the two-phase broadcast, detects crashes and takes part in the elections. `Client` sends read and write requests to a replica and sets a timeout on each of them. `NetworkChannel` is the FIFO channel actor supplied with the codebase: one instance per (sender, destination) pair, it delays every message by a random amount between 5 and 20 ms while preserving order, which emulates network propagation delay. All replica-to-replica and replica-to-client traffic goes through `AbstractReplica.tell(msg, dst)`, which routes the message through the proper channel.

`AbstractReplica` and `AbstractClient` come from the mandatory codebase and were not modified. They provide the crash command, the initialization message, the logging helpers and the callbacks thath the automated tests observe.

Packages

`it.unitn.ds`	`Replica, Client, Main, UpdateID, Update, UpdateHistory`
`it.unitn.ds.messages`	`all protocol messages, one class each`
`it.unitn.ds.election`	`RingTopology, ElectionLogic, SyncPlan (pure logic)`

The `election` package deserves a note. It contains no actor code and no mutable state: every method is a pure function of its arguments (the ring order, the winner of an election payload, the list of updates to replay). This keeps the decision rules of the election separate from the message plumbing and makes the unit-testable without starting an actor system.

Data classes

`UpdateID` is the immutable pair $\langle epoch, sequence \rangle$ that identifies an update. It implements `compareTo` as the lexicographic order, on $(epoch, sequence)$, so "more recent" is simply "greater", and it implements `equals/hashCode` so it can be used as a map key. `Update` bundles an `UpdateID` with the pair $(index, value)$ to write.

`UpdateHistory` is the append-only log of the updates a replica has delivered. It exposes `latestId()`, used to publish how up to date a replica is during an election, and `after(threshold)`, which returns the entries strictly newer than a given id and is exactly the diff a lagging replica needs after an election. Entries can only be appended, nwver removed or reordered.

Messages

Every message is a separate class, is `Serializable`, and has only `final` fields; the collections carried by `Election` and `Synchronization` are wrapped in unmodifiable views before being sent. No mutable state is ever shared between actors.

Phase	Message	Direction
Client	`ClientRead`, `ClientWrite`	client → replica
	`ReadReply`, `WriteReply`	replica → client
Update	`ForwardWrite`	replica → coordinator
	`UpdateMsg`	coordinator → all
	`UpdateAck`	replica → coordinator
	`WriteOk`	coordinator → all
Failure det.	`Heartbeat`	coordinator → all
	`HeartbeatTimeout`, `UpdateTimeout`	self
	`ForwardTimeout`	
Election	`Election`, `ElectionAck`	replica → replica
	`Synchronization`	new coordinator → all
	`ElectionAckTimeout`, `GlobalElectionTimeout`	self

All timeouts are self-messages scheduled with the Akka scheduler, so they enter the mailbox like any other messages and are processed by the behaviour that is active when they fire.

2 System Design

Replica states

A replica is always in one of three behaviours, switched with `getContext().become(...)`:

- **NORMAL** — the two-phase broadcast and crash detection.
- **ELECTION** — a round is in progress. Reads are still served locally, incoming client writes are buffered, and all messages of the dying epoch (`UpdateMsg`, `WriteOk`, heartbeats and their timeouts) are ignored.
- **CRASHED** — every message is dropped.

The transitions are: NORMAL → ELECTION when a crash of the coordinator is detected or an `Election` message arrives; ELECTION → NORMAL when the round ends, either because this replica won or because a `Synchronization` was received; and anything → CRASHED when a crash is triggered.

Ignoring updates while in ELECTION is what implements Hint 1 of the assignment: the "latest known update" a replica pusblishes must not change while it is a candidate, otherwise two replicas could compare payloads built at different moments. The crashed behaviour is a receive that matches anything and does nothing; the actor is deliberately *not* stopped, so that senders keep seeing a silent node rather than dead letters.

Update protocol

A read is answered immediately by the contacted replica from its local array. A write is forwarded to the coordinator (oe handled directly, if the contacted replica is the coordinator). The coordinator assigns the next id $\langle e, i \rangle$ in the current epoch and broadcasts an `UpdateMsg`. Every replica stores the proposal as pending and replies with an `UpdateAck`. When the coordinator has collected $|Q| = \lfloor N/2 \rfloor + 1$ acknowledgements it broadcasts `WriteOk`, and each replica then applies the update to its array, appends it to its history and, if it is the replica the client contacted, sends the `WriteReply`.

The coordinator is a replica like the others and sends both broadcasts to itself as well, so it applies the update through its own `WriteOk` and its own acknowledgement counts towards the quorum, as the assignment requires.

Total order follows from three facts: within an epoch there is a single coordinator, which is the only source of `UpdateMsg` and `WriteOk`; channels are FIFO per sender–receiver pair, so every replica sees that coordinator's `WriteOk` messages in the order they were sent; and across epochs the epoch number is strictly increasing, so an update of epoch $e' > e$ in the $\langle epoch, sequence \rangle$ order.

Because reads are served locally without contacting the coordinator, a client that writes and then immediately reads from the same replica may still observe the old value: the write has to complete a round trip through the coordinator before it is applied anywhere. This is sequential consistency from the point of view of a client that always talks to the same replica, which is what the project requires, not linearizability.

Crash detection

The coordinator broadcasts a `Heartbeat` every second. A replica suspects it in three situations, exactly the ones listed in the assignment:

Timeout	Set when	Value
`HeartbeatTimeout`	always, on a non-coordinator	3×1000 ms
`UpdateTimeout`	an `UpdateMsg` was ack'd	3×1000 ms
`ForwardTimeout`	a write was forwarded to the coord.	3×1000 ms
`ElectionAckTimeout`	an `Election` was forwarded	$20 + 10N$ ms
`GlobalElectionTimeout`	the replica entered ELECTION	$2N \times (20 + 10N)$ ms

The heartbeat watchdog is reset by *any* message coming from the coordinator, not only heartbeats: an `UpdateMsg` or a `WriteOk` is just as good a proof of life. The first three timeouts are three beats long, which is generous with respect to the 5–20 ms simulated latency; the project assumes crash detection is accurate and does not produce false positives, so we preferred slow detection over the rist of suspecting a live coordinator. The two election timeouts are instead scaled on `getMaxLatencyPlusTolerance()`, which already grows with N, because they bound one hop and one full ring lap respectively.

Coordinator election

The ring is the replica ids in ascending order; the successor of a replica is the next id, wrapping around, skipping every id in a local `suspected` set. That set starts with the coordinator believed to be dead and grows with every successor that fails to acknowledge.

A replica that detects the crash enters ELECTION and sends `Election` message carrying a map $id \mapsto latestId$ containing its own entry, where the value is the id of its most recent delivered update, or the sentinel $\langle 0, 0 \rangle$ if its history is emtpy. Each replica that receives the message acknowledges the sender immediately, adds its own entry if not already present, and forwards it to its successor.

When a replica receives a payload that already contains its own id, the message has completed a full lap and the payload is final. The winnder is then computed locally as the replica with the greatest latest id, ties broken by the greatest replica id. Since the payload is the same for everybody every replica computes the same winner without any further round. A replica that recognises itself as the winner takes over; a replica that is not the winner simply keeps the message circulating, so that it reaches the winner.

Crashes during the election are handled by the hop-by-hop acknowledgement: the sender of an `Election` sets an `ElectionAckTimeout`, if it expires, adds the silent successor to `suspected` and re-sends the same payload to the following replica in the ring. Because a strict majority is always alive, this always, terminates on a live replica, and at least one live replica necessarily holds the most recent update.

What the new coordinator does

The winner performs the following steps, in this order:

1. it completes the interrupted updates — every phase-1 proposal still pending in its own state is applied now (Hint 3);
2. it sets its next id to $\langle e_{new}, 0 \rangle$, where e_{new} is one more than the highest epoch appearing in the election payload, so the sequence number restarts from 0 at every epoch change;
3. it broadcasts a `Synchronization` carrying the new epoch and the updates the others may be missing;
4. it starts emitting heartbeats and returns to NORMAL.

Step 1 must precede Step 3, otherwise the recovered updates would not be part of the announcement. This is what makes uniform agreement hold: if the old coordinator died with the `WriteOk` delivered to a single replica, that replica is the only one holding the update, and it is exactly the one that wins the election — so the update is put back into circulation instead of being lost with the old epoch. The list to replay is computed once, as the diff between the winner's history and the *most lagging* participant of the election, so that a single broadcast brings everybody up to date; replicas that already delivered part of it skip those entries, since delivery is idempotent on the update id.

The replicas that receive `Synchronization` adopt the new coordinator and epoch, apply the updates they are missing, discard whatever is left of the dead epoch and go back to NORMAL.

Making sure the election terminates

The dangerous case (Hint 2) is the winner crashing after the payload has completed its lap but before the `Synchronization` is sent: the message dies with it and everybody else waits forever. For this reason every replica that enters ELECTION also sets a `GlobalElectionTimeout`, long enough for a full ring lap in which every hop times out. When it fires, the replica simply starts a fresh round. Since the dead winner is by then suspected by at least its predecessor, and suspicion only grows, the second round converges on a live replica.

A second, subtler termination problem is a late `Election` message from a round that has already been decided. Processing it would drag a settled system back into an election. Such a message is recognized because it would elect the coordinator we are already following, and is dropped; if we are that coordinator, we answer with a `Synchronization` so that the sender, which is evidently still stuck in the old round, can move on.

3 Implementation

Storing updates

Each replica keeps three things: the array `positions`, an `UpdateHistory` with the updates it has delivered, and a map from `UpdateID` to the phase-1 proposals that are not confirmed yet. The coordinator also tracks the lst id assigned, how many acks each id has collected, and the set of ids that already reached the quorum. The last set is what stops a second `WriteOk` from being sent for the same update when late acks keep arriving.

Delivery happens in one place, `applyUpdate`, which wirtes the array, appends to the history, fires the mandatory callback and answers the client if this replica is the one that was contacted. Every path that can deliver — `WriteOk`, the recovery of interrupted updates, and the `Synchronization` — goes through it, and each of them first checks that the id is not already in the history. This is what guarantees integrity: an update is applied at most once, even when the same update arrives both as a `WriteOk` and inside a synchronization.

Managing timeouts

Timers are `Cancellable` objects returned by the scheduler. Since the same kind of timeout can be in flight for several updates at once, the per-update ones are kept in maps rather than in a single field: `updateTimeouts` keyed by `UpdateID` and `forwardTimeouts` keyed by the client request id. Using the request id rather thatn the $(index, value)$ pair matters, because two distinct requests may write the same value at the same index and would otherwise be indistinguishable.

The rule we follow everywhere is *cancel before re-setting*: scheduling a new timer over a variable that still holds a live one makes the old timer impossible to cancel afterwards, and it would eventually fire and trigger a spurious election. Every timer is cancelled as soon as the message it was waiting for arrives: the `UpdateTimeout` on the corresponding `UpdateMsg` shows up, the `UpdateTimeout` on the corresponding `WriteOk`, the `ElectionAckTimeout` on the `ElectionAck`. A crashing replica cancels all of its timers, so that it also stops sending.

Controlling crashes

A crash is initiated from the outside with `Crash(type, n)`, whose semantics are "process n messages of that type, then crash". The counter is kept per type in `messageCounters` and checked by `checkCrashCondition`, which is called at the top of the handlers for `UpdateMsg`, `WriteOk`, `Heartbeat` and `Election`, and returns `false` when the replica has just died so the handler can stop. `Crash.Type.Now` crashes immediately.

The same counter is also evaluated inside `broadcast`, once per recipient and before each send. This is what makes a *partial* broadcast possible: `Crash(WriteOk, 2)` on the coordinator means it hands the `WriteOk`, to exactly two replicas and then dies, leaving the others without it. There is no ambiguity between the two uses of the counter, because for any given message type a replica is either the one broadcasting it or one of the receivers, never both. The broadcast walks the group in ascending id order so that the set of replicas that were served is reproducible from one run to the next, which is what makes the corner-case tests deterministic.

Crashing means switching to a behaviour that ignores everything; `getContext().stop()` is never called. Messages already handed to a `NetworkChannel` are still delivered, since the channels are not stopped: a crash prevents *starting* new sends, it does not recall messages already in flight.

Client writes across an election

The supplied client sends each request exactly once and reports a timeout if no answer arrives; it does not retry. A write whose coordinator dies mid-flight would therefore simply be lost. To avoid this, the contacted replica keeps every unanswered client write in `clientWrites` and replays it to the new coordinator as soon as the election ends. Writes that arrive while the replica is already in ELECTION are buffered on arrival for the same reason. THe nrey is removed then the corresponding `WriteReply` is finelly sent.

One consequence needed some care: the new coordinator answers every late `Election` with a `Synchronization`, so the same announcement can be received more than once. Running the handler twice would replay the buffered writes a second time and turn one client request into two updates, so a `Synchronization` that we have alreay adopted — same coordinator, same epoch, and we are not in an election — is recognized and ignored.

Testing

The suite has 98 tests, al passing. Besides the tests supplied with the codebase (`NoCrashes`, `WithCrashes`, `APICompliance`), there are pure unit tests for the three classes of the `election` package, which run without an actor system, and and end-to-end suite `scenarios/CornerCases` that drives the crash instrumentation into the dive interesting points of the protocol: coordinator dying during the `UPDATE` broadcast; coordinator dying with the `WRITEOK` only partially disseminated; two *consecutive* ring neighbours crashing, so the election has to skip twice in a row; the winner crashing before announcing itself, which only the `GlobalElectionTimeout` can resolve; and a replica dying after its ack but before applying, which must not disturb the epoch.

`Main` contains four demo scenarios (`./gradlew run`), each with its own actor system so that its log can be read on its own.

Additional assumptions

1. **Messages already queued survive a crash.** The channel actors of a crashed replica are not stopeed, so what was already handed to them is still delivered.
2. `ElectionAck` **carries no correlation id.** The handler cancels the current timer without checking who sent the ack. A late ack from an already skipped successor can therefore cancel a timer just reset for another once; the worst effect is a slower round, covered by the `GlobalElectionTimeout`.
3. **The winner also delivers updates that never reached a quorum.** Recovery applies every pending phase-1 proposal, including those with fewer than $\lfloor N/2 \rfloor + 1$ acks. This is the conservative choice — complete rather than discard — and it violates none of the four properties, since all correct replicas converge on the same state through the synchronization.
4. **Reads during an election are served locally** and may return a value that is about to be superseded. This is sequential consistency, which is what the project asks for.
5. **Client-to-replica traffic does not go through the simulated channel.** `AbstractClient` does not expose the channel helper, so that one direction has no artificial delay; replica-to-replica and replica-to-client traffic does.
6. **The buffer of client writes is unbounded.** With short elections and test clients this is not a concern, but a very long election could make it grow without limit.
7. **Membership is static and the** `suspected` **set only grows**, consistently with replicas that fail by crashing and never recover.